BOSTON UNIVERSITY / CS 528 CLOUD COMPUTING

Homework 5
Cloud SQL Request Logging & Performance Analysis

Implementation, deployment instructions, database schema, functional verification, request analytics, and timing instrumentation.

Submission detail	Value
Student	Zukhriddin Fakhriddinov
GCP Project	quick-formula-457319-q3
Region / zone	us-central1 / us-central1-a
Repository	github.com/zukhriddingit/cs528-hw4-vm-services/tree/main/hw5
Report prepared	2026-10-27 23:57 UTC
Evidence status	Full workload statistics - check expected count below

Evidence integrity: All measured figures in this report are from the saved course-client experiment. If the full run has not completed, pilot measurements are explicitly identified and must not be presented as 100,000-request results.

1. System architecture and continuity from HW4

The HW4 Flask/Gunicorn file-serving application was extended to persist HTTP request metadata to PostgreSQL in Cloud SQL. HTML content continues to be served from the existing Google Cloud Storage bucket, and the HW4 Pub/Sub reporter (second application) is kept unchanged. Each incoming request creates a row in **requests**; requests with non-200 status additionally create a row in **failed_requests**.

Component	Implementation
HTTP web VM	hw5-server; Compute Engine e2-standard-2, Debian 12, Gunicorn/Flask, TCP 8080
Reporter VM	hw5-reporter; e2-micro, same HW4 Pub/Sub subscriber source
File storage	gs://cloud_hw2_bucket/pages/[file].html
SQL persistence	hw5-sql; PostgreSQL 16 Enterprise, db-f1-micro, 10 GB SSD, ZONAL
SQL connection	Cloud SQL Python Connector (pg8000), SQLAlchemy pool; credentials via Secret Manager
Forbidden reporting	X-country denylist; Pub/Sub topic hw3-forbidden-requests; reporter subscription
Safety mechanism	Authenticated Gen2 Cloud Function + hourly Cloud Scheduler job
Clients	Two concurrent instances of the provided macOS ARM64 client, seed 5285

Request processing and authorization

The web app extracts X-country, X-client-IP, X-gender, X-age, X-income, and X-time, plus the requested filename. Banned countries are rejected with HTTP 400 and forwarded to the existing Pub/Sub reporting path. Existing pages return 200; missing pages return 404; unsupported methods return 501. An atomic SQL transaction inserts the main request row and, for every non-200 response, an associated failure row.

The request country/IP/age/income fields are synthetic values provided by the course client, not verified real-world demographic or geolocation data.

2. Cloud SQL schema

The database is **hw5_requests**, owned/initialized by PostgreSQL user **hw5_user**. Schema creation uses CREATE TABLE IF NOT EXISTS so reruns do not destroy previously logged requests.

Table	Column	PostgreSQL type / purpose
requests	request_id	BIGINT IDENTITY, PRIMARY KEY
	country, client_ip	TEXT, INET
	gender, age, income	VARCHAR(16) category labels
	is_banned	BOOLEAN NOT NULL
	time_of_day	TIMESTAMP WITHOUT TIME ZONE
	requested_file, status_code	TEXT NOT NULL; SMALLINT NOT NULL

Table	Column	PostgreSQL type / purpose
	received_at	TIMESTAMPTZ NOT NULL; DEFAULT CURRENT_TIMESTAMP
failed_requests	failure_id	BIGINT IDENTITY, PRIMARY KEY
	request_time	TIMESTAMPTZ NOT NULL
	requested_file, error_code	TEXT NOT NULL; SMALLINT NOT NULL, error_code != 200

The failure table contains only the three required event attributes plus its surrogate primary key. The application records timestamps in UTC and preserves user-provided age and income bins as text.

3. Deployment, startup, and teardown instructions

The grader should use the lowercase **hw5/** directory. setup.sh and cleanup.sh hardcode the project ID in _common.sh, independent of the grader's active gcloud project. A VM startup script is installed using --metadata-from-file and is guarded with a run-once package installation lock. No gcloud compute ssh --command is used for VM provisioning.

Action	Command from repository root
Provision	bash hw5/setup.sh
Check VM state	gcloud compute instances list --project=quick-formula-457319-q3
Resolve IP	gcloud compute addresses describe hw5-server-ip --region=us-central1 --format="value(address)"
Curl verification	bash hw5/verify_curl.sh
Run 2 x 50,000 requests	caffeinate -i bash hw5/run_load.sh 50000
Analyze saved evidence	python3 hw5/generate_report.py
Teardown	bash hw5/cleanup.sh

Infrastructure behavior. setup.sh creates or reuses the SQL instance, user, and schema; provisions/reuses service accounts, bucket read permission, Cloud SQL Client access, Pub/Sub publisher/subscriber access, Secret Manager secret access, web VM and reporter VM, allocated static IP, firewall, Gen2 stop function, and Scheduler. It retrieves the allocated IP with gcloud rather than hardcoding an address.

Startup behavior. startup.sh runs as root on each VM boot. Its lock file prevents repeating apt/pip installation, while source/metadata/service updates still occur on subsequent boots. A Gunicorn web service launches automatically under www-data.

Cleanup behavior. cleanup.sh stops but does not delete Cloud SQL, removes the HW5 VMs and static IP, and removes the HW5 firewall/subscription/function/scheduler. Legacy HW4 resources and shared HW3 Pub/Sub topic are preserved. No gcloud auth application-default login is used, so ADC revocation is unnecessary.

Hourly cloud database safety function

An authenticated Cloud Scheduler POST request triggers the Gen2 function **hw5-sql-guard** at minute 0 of each hour (UTC). The function reads the instance state through the Cloud SQL Admin API and patches activationPolicy to NEVER if the database is RUNNABLE. During long experiments, run_load.sh temporarily pauses this job; its exit trap attempts to stop SQL and resume the job. Network outages can prevent cleanup, so final cloud state must be verified manually if errors occur.

4. Curl verification: table contents before and after each request

On October 27, 2026, the three requested curl demonstrations returned the expected HTTP codes. The table contents were queried immediately before and after each request using inspect_tables.py. Counts below are directly supported by the saved terminal output.

Case	HTTP	requests before -> after	failed_requests before -> after
GET /0.html (USA)	200	0 -> 1	0 -> 0
GET /999999.html (USA)	404	1 -> 2	0 -> 1
GET /0.html (Iran)	400	2 -> 3	1 -> 2

Recorded rows: request_id 1: file 0.html, USA, status 200, is_banned=false. request_id 2: file 999999.html, USA, status 404, with failed_requests failure_id 1. request_id 3: file 0.html, Iran, status 400, is_banned=true, with failed_requests failure_id 2. The first three request IDs and status codes are verified by the provided terminal transcript.

The reproducible plain-text transcript is saved under hw5/evidence/curl_*.txt.

Screenshot requirement: Attach actual terminal screenshots from the curl run in hw5/evidence/screenshots/. Images found there are appended to this PDF; textual transcript summaries alone are not literal screen captures.

5. Concurrent-client load test and database statistics

Full 100,000-request load experiment. Both client processes use the provided course executable with identical random seed 5285. The launch wrapper snapshots the starting request ID and filters all statistics to (start_id, end_id]. This avoids including the earlier curl tests or unrelated traffic.

Evidence directory: vm_load_20261027T220630Z_50000x2

100,000	95,403	4,597	4,597
Client requests recorded	HTTP 200	Non-200	Banned countries

Reconciliation: 100,000 / 100,000 expected requests appear in the database under the course-client header filter. Other rows in ID range: 16.

HTTP status codes and gender split

HTTP status	Requests
200	95403
400	4597

Gender	Requests
Male	50013
Female	49987

Five most frequent source countries

Country	Requests
Ethiopia	570
Palestine	568
Botswana	562
Morocco	559
Syria	557

Age and income frequency

Age group	Requests
26-35	12661
17-25	12594
56-65	12578
46-55	12534
36-45	12472
66-75	12472
76+	12390
0-16	12299

Income group	Requests
10k-20k	12606
60k-100k	12570
40k-60k	12544
20k-40k	12540
100k-150k	12525
0-10k	12490
150k-250k	12442
250k+	12283

Most active age group: 26-35 (12,661)
Most active income group: 10k-20k (12,606)

A response code of 200 is counted as successful; every other HTTP status is counted as unsuccessful. Network failures that never reached the web server cannot be represented in these tables.

6. High-resolution web server timing results

The Flask service records request-stage durations with Python time.perf_counter_ns(). Header extraction, GCS download, SQL transaction, and response construction reside in separate functions. A wrapper around the WSGI iterator records the response-send interval and total server processing duration. Timing records are emitted to a CSV on the web VM and retrieved after the run.

Measured request timing rows: **100,016**. Values are from the selected full-run timing CSV.

Measured stage	Mean (ms)	Median (ms)	P95 (ms)
Header extraction	0.049	0.043	0.070
Cloud Storage read	40.695	32.280	87.995
Cloud SQL insert/transaction	35.520	11.813	195.417
Response preparation	0.033	0.029	0.047
WSGI response-send interval	0.615	0.601	1.272
Total server processing	82.914	49.278	243.971

P95 is the sample 95th percentile using a nearest-rank-style indexed observation. The WSGI send interval approximates socket write/iterator consumption time inside the server; it is not a measurement of full client-side download completion. Timed stages may exclude ancillary logging and overhead, so stage means need not sum precisely to total processing time.

7. Resource usage and cloud billing

The reported infrastructure consists of the web VM (e2-standard-2), reporter VM (e2-micro), PostgreSQL Enterprise (db-f1-micro / 10 GB), Cloud Storage objects/operations, Pub/Sub, Cloud Scheduler/Functions, disk and external IP resources, and outbound data transfer. Database compute is stopped when not in use; storage and some network/IP charges may remain.

Actual billed spend: awaiting finalized Google Cloud Billing data. The assignment explicitly permits submitting **billing_repord.pdf** to the GitHub repository the following day because cloud billing data can be delayed. No unverified monetary amount is claimed here.

Billing reproducibility: open Google Cloud Billing, select project quick-formula-457319-q3, set the custom date range to cover HW5 development/execution, and group charges by service. Include any costs related to these resources and note that earlier HW4 resources might also generate charges if left allocated.

8. Reproducibility and evidence files

Submission item	Location
First app (modified)	hw5/service1/app.py
Second app (unchanged from HW4)	hw5/service2/main.py
Stop function	hw5/sql_monitor/main.py
Database schema	hw5/schema.sql and hw5/setup_schema.py
Automated deployment/cleanup	hw5/setup.sh; hw5/startup.sh; hw5/cleanup.sh
Load test, statistics, timing scripts	hw5/run_load.sh; hw5/request_statistics.py; hw5/analyze_timings.py
Saved curl logs	hw5/evidence/curl_*.txt
Load-run outputs	hw5/evidence/load_*/request_statistics.json; timings_run.csv
Cloud billing addendum	billing_repord.pdf (once cloud billing data is final)

Repository: github.com/zukhriddingit/cs528-hw4-vm-services/tree/main/hw5. Verify that the HW5 files have actually been committed and pushed to the default branch before submitting this link.

Note: the dataset is synthetic course traffic and should not be interpreted as actual users' nationalities, ages, incomes, or IP locations.